

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

"Forty-two," said Deep Thought, with infinite majesty and calm.
Douglas Adams, *The Hitchhiker's Guide to the Galaxy*, 1979

Object Lifecycle

*T*his chapter discusses the lifecycle of objects and the reasons why state changes can be meaningful to the application developer. A closer look is taken at mandatory and optional states of instances, the methods to gather information about states and methods that lead to state transitions.

Why are object lifecycles and state transitions important to know? Imagine what happens inside of a Java VM whenever 'new' is called by an application: A small portion of memory needs to be allocated, a reference is returned to the application, but behind the scenes, the world is more complex. First, the Java VM might load the byte code from some resource, compile the byte code to native machine instructions, static initialization takes place, the base classes are loaded, and constructors are called. Even more complex is the destruction of memory instances due to garbage collection, weak references, and queues and finalize callbacks. There must be some well-defined life cycle model inside the Java VM, which lets Java objects to be used predictably and without "Deep Thought".

4.1 INSTANCE LIFECYCLE

The same question about "life" can be applied to persistent instances: Knowing about the object's life cycle is not inevitably needed, especially in every little detail, but whenever callbacks or other reentrant code is used, it is strongly recommended to think about the side effects of state transitions.

JDO defines seven required and three optional states: transient, persistent-new, persistent-new-deleted, hollow, persistent-clean, persistent-dirty and persistent-deleted are the mandatory life-cycle states. Transient-dirty, transient-clean and persistent-non-transactional are optional states and are explained later in this chapter. Some of the state transitions are directly invoked by the application itself, for instance making an object

persistent. Starting or terminating a transaction can also trigger an object's state change, which might be invoked directly by the application developer through a call to Transaction methods begin(), commit() or rollback(). Note that in managed environments state changes can happen due to an external transaction controller.

Mandatory and optional JDO instance states

JDO mandates seven states:

hollow
persistent-clean
persistent-dirty
persistent-new
persistent-deleted
persistent-new-deleted
transient

Optional states are:

transient-dirty
persistent-non-transactional
transient-clean

A persistence-capable instance can enter the JDO object life cycle by two operations: A call to makePersistent or an instantiation by the JDO vendor's implementation when an existing object is loaded from a data store. The following sections explain two typical state transitions: making objects persistent and re-creating objects from the data store.

4.1.1. Making Objects Persistent

4.1.1.1. Transient

Instances of persistence-capable classes behave as any other Java objects when they are created by the 'new' operator. Field access is not mediated for these objects resulting in almost the same access performance as for unenhanced classes. Transient instances are not influenced by transactional methods like begin, commit or rollback except when they are reachable by already persistent objects.

4.1.1.2. Persistent-New

A call to PersistenceManager.makePersistent assigns a new object id to the instance and the state changes to persistent-new. A persistent-new object behaves like a transient object, but it will be flushed to the data store at some time. Any objects, which are reachable through persistent fields, are made persistent and so on, until all reachable objects are either of a transient type or persistent-new. Though reachable objects are provisionally persistent, their state is called persistent-new. Provisionally persistent-new objects behave like persistent-

new objects, but their state may change during commit, if the provisionally persistent object isn't reachable anymore.

4.1.1.3. Hollow

After successful completion of the transaction, the persistent-new objects' state changes to hollow. A hollow object is just a reference and represents some data in the data store, which is not yet in memory or outdated. Usually other clients or applications are free to access or change data referenced by hollow objects, for instance in a different transaction, process or Java VM. Additionally, the contents of such hollow objects are cleared after commit, to free memory. If references were kept between hollow objects, the garbage collector may not free a graph of objects. After some transactions, a bunch of hollow objects might be accumulated. The hollow state cannot be interrogated by the application. There is no JDOHelper method and access to fields may result in a state change, so that it is impossible to tell the previous state.

4.1.1.4. Persistent-new deleted

Whenever persistent-new objects are deleted by a call to PersistenceManager.deletePersistent, their state changes to persistent-new deleted. There is no way to undelete an object, which means this state can be left only at transaction termination. Accessing a field of a deleted object results in a JDOUserException, except when reading primary key fields. After transaction termination, (commit or rollback) the deleted instance's state changes back to transient.

4.1.2. Creating objects from the data store

4.1.2.1. Hollow

Hollow is also the initial state for objects that are firstly created to represent a reference, but no valid data is read yet. Any field access is checked and data is retrieved on demand. Objects having the hollow state are instantiated by calls such as Extent.iterator().next(), PersistenceManager.getObjectById() or query execution. Usually, persistent fields of hollow objects are initialized to the field's default values (zero or null), but since persistence-capable instances are created through their no-args constructors, they may be filled with other values as well. This might be the only visible difference to objects, which became hollow after they have been inserted in a previous transaction as mentioned in the preceding section.

4.1.2.2. Persistent-clean

Objects transition from hollow to persistent-clean after they have been filled with valid data from the data store, but the data has not yet been modified in the current transaction. The state is called clean, because the object's data is consistent with the corresponding data in the data store. While an object is filled, field references to other objects are resolved by the PersistenceManager. Either those references may point to objects already in memory, or the PersistenceManager creates new hollow objects. Persistent-Clean objects might be collected by the Java VM's garbage collector, if the application does not refer the objects anymore. The PersistenceManager is required to cache the references to objects and instantiate new hollow objects in case the objects were already garbage-collected.

4.1.2.3. Persistent-dirty

Objects transition from either persistent-clean or hollow to persistent-dirty when their transactional data is changed. This state is called dirty, because the in-memory data of the object is inconsistent with the persistent data corresponding to the object in the data store. The application should not depend on modification tracking, if the value assigned to a managed field is equal to the previous value. Furthermore, an instance will not change back to persistent-clean or hollow, when previous field values or modifications are restored. Whenever transient objects are assigned to persistent-clean or persistent-dirty objects, they do not transition to persistent-new at that moment.

4.1.2.4. Persistent-deleted

In a call to `PersistentManager.deletePersistent`, a persistent instance changes its state to persistent-deleted. It is impossible to leave this state except by terminating the transaction, which means there is no method to undo a `deletePersistent` call except rollback. A successful transaction termination makes a deleted instance transient. An object cannot be made persistent again with the same identity as long as it is deleted.

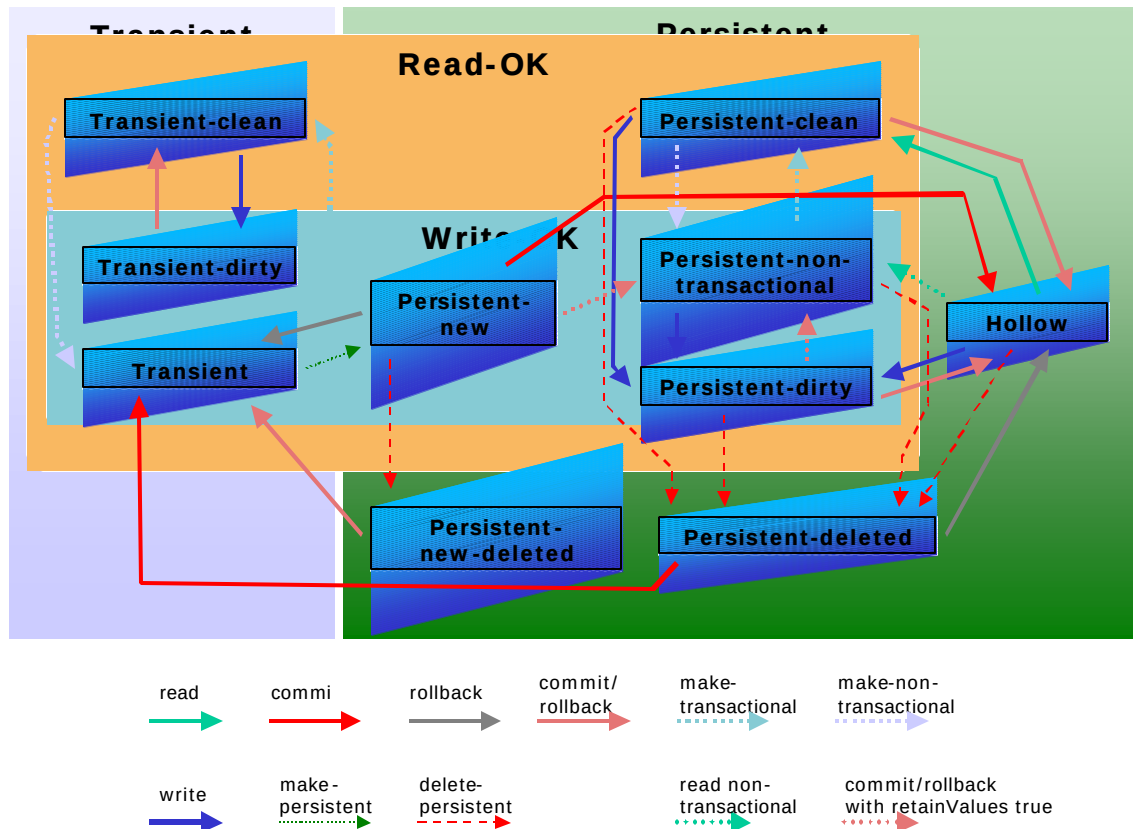


Figure 4-1 Life Cycle: State diagram

4.2 HOW TO FIND OUT ABOUT STATE

The previously mentioned seven states transient, persistent-new, persistent-new-deleted, hollow, persistent-clean, persistent-dirty and persistent-deleted are mandatory for all JDO implementations though there is no API to get the object's state. The exact state of a JDO instance is known only by the JDO implementation and some states might not even be implemented by some vendor. The JDO states are more a kind of behavioral definition. Nevertheless, some methods let us interrogate the characteristics of an object and draw conclusions about the object's state.

These methods are in the JDOHelper class:

```
boolean isDeleted(Object o)
boolean isDirty(Object o)
boolean isNew(Object o)
```

```
boolean isPersistent(Object o)
boolean isTransactional(Object o)
```

There is no method to find out, whether an object is hollow. It is a vendor's choice how to handle the lazy on-demand reading of fields and objects. Later in this chapter, another way is shown to get information about an object's hollow state.

```
public class Example01
{
    public void info(String msg, Object obj)
    {
        String spersistent = JDOHelper.isPersistent(obj) ?
            " persistent" : "";
        String snew = JDOHelper.isNew(obj) ?
            " new" : "";
        String sdirty = JDOHelper.isDirty(obj) ?
            " dirty" : "";
        String sdeleted = JDOHelper.isDeleted(obj) ?
            " deleted" : "";
        String stransact = JDOHelper.isTransactional(obj) ?
            " transactional" : "";
        System.out.println(msg + sdeleted + sdirty + snew +
            spersistent + stransact);
    }
}
```

Some lines of code illustrates the states and methods up to now.

```
public static main(String argv[])
{
    PersistenceManager pm = pmf.getPersistenceManager();
    pm.currentTransaction().begin();

    Book book = new Book();
    info("a new book ", book);

    pm.makePersistent(book);
    info("persistent book ", book);

    pm.deletePersistent(book);
    info("deleted book ", book);

    pm.currentTransaction().rollback();
}
```

```

        info("rolled back ",book);

        pm.currentTransaction().begin();
        pm.makePersistent(book);
        pm.currentTransaction().commit();
        info("new commit ",book);

        pm.currentTransaction().begin();
        String title = book.title;
        info("read field ",book);

        String title = "new "+book.title;
        info("modified field ",book);

        pm.deletePersistent(book);
        info("deleted book ",book);

        pm.currentTransaction().commit();
        info("last commit ",book);
    }
}

```

The code simply assembles the strings for the boolean state functions and the main method performs some state transitions. When the example is run, it prints the following:

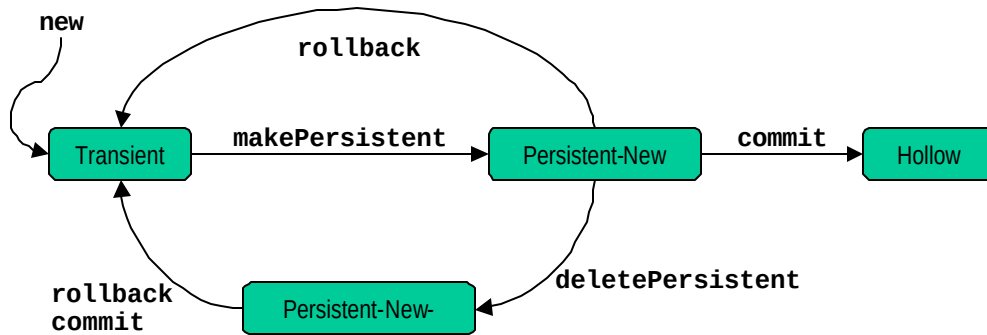
```

a new Book: dirty
persistent book: persistent new dirty
deleted book: persistent new dirty deleted
rolled back: dirty
new commit: persistent
read field: persistent
modified field: persistent dirty
deleted book: persistent dirty deleted
last commit: dirty

```

4.3 OPERATIONS THAT CHANGE STATE

Some methods alter the object's state by a direct call, but other indirectly change the state of multiple objects. Here is a list of operations that modify an object's state:



4.3.1. makePersistent

The PersistenceManager method makePersistent causes transient objects to become persistent-new. In a call to makePersistent the object graph is traversed to find other reachable, yet transient objects. Those objects may indirectly become persistent-new as well.

Figure 4-2 Life Cycle: New instances

4.3.2. deletePersistent

A call to deletePersistent makes clean, dirty or hollow objects persistent-deleted whereas persistent-new objects become persistent-new-deleted. This operation does not involve other reachable objects.

4.3.3. makeTransient

An object can be made transient, if the previous state is persistent-clean or hollow. The persistent object is detached from the PersistenceManager and loses its identity. This PersistenceManager method does not involve other reachable objects.

4.3.4. commit

All new, clean and dirty objects become hollow at commit time. All other objects become transient. After commit, there are either hollow objects associated with a PersistenceManager or regular, transient Java objects.

4.3.5. rollback

Clean, dirty and deleted objects become hollow at rollback. Other objects become transient. As a rule of thumb, hollow and transient objects are set back to their previous state after rollback.

4.3.6. refresh

A call to the PersistenceManager method refresh reloads the object's data and any modifications are lost. Persistent-dirty objects will be Persistent-clean after refresh.

4.3.7. evict

Evict helps the PersistenceManager caching implementation to save memory. Persistent-clean objects that are evicted become hollow. The fields of the objects are cleared to allow garbage collection of unreferenced sub-objects.

4.3.8. read access to fields

After reading the data from the data store, hollow objects become persistent-clean. If pessimistic concurrency control is used, the object may be locked against concurrent modifications.

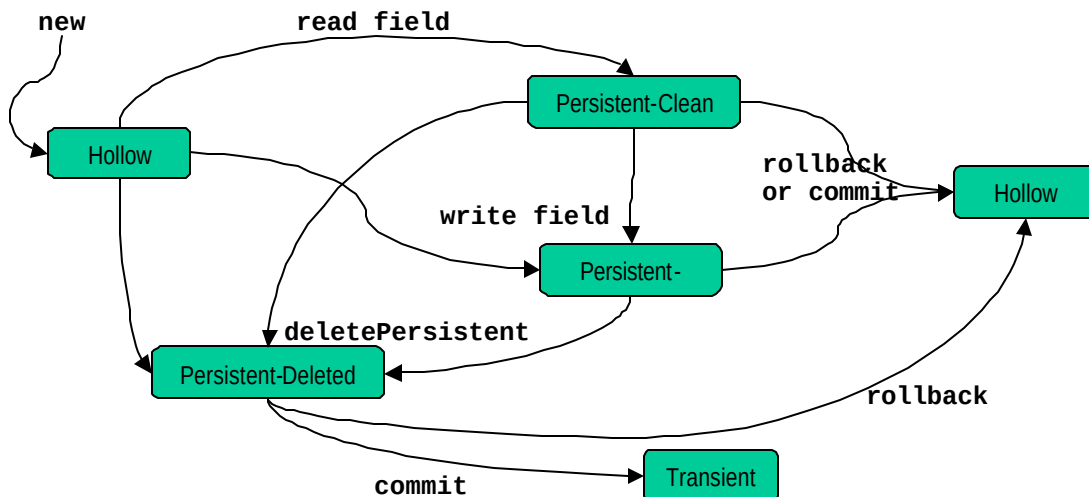


Figure 4-3 Life Cycle: Creating objects from the data store

4.3.9. write field inside a transaction

When a field of an object, which is attached to a `PersistenceManager`, is modified inside of a transaction, the persistent object has to be remembered by the JDO implementation to later update the object's data in the data store. A persistent-clean or hollow object becomes persistent-dirty in this case. If pessimistic locking is used, a JDO implementation might also lock the object against access by other clients of the same data store. Field modification of transient objects or objects that are persistent-dirty or persistent-new has no effect on the object's state. Whenever another object is assigned to a field, the modified object is marked persistent-dirty, but the referenced object is kept unchanged.

4.3.10. retrieve

The `PersistenceManager.retrieve()` method basically performs the same operation as reading of a field inside of an active transaction.

4.4 CALLBACKS

By implementing callback methods, an application can intercept some of the above life cycle events. There are three major cases where an application developer might wish to implement some special behavior of persistent classes:

- Read or write fields that are not persistence-capable or should not be stored by the default JDO algorithm
- Delete dependent sub-objects
- Clear unneeded references

The callback methods are placed in an extra interface, which is handled like the `java.io.Serializable` tag interface. If a persistence-capable class implements the `InstanceCallbacks` interface the callback methods are called by the JDO implementation, otherwise they are not called.

InstanceCallbacks

If a class implements the `InstanceCallbacks` interface, the methods `jdoPostLoad`, `jdoPreStore`, `jdoPreClear` and `jdoPreDelete` are automatically called. Compared to the `Serializable` interface, a class is responsible to call its super class `InstanceCallbacks` methods.

4.4.1. Uses of jdoPostLoad

The `jdoPostLoad` method is called whenever a persistent object is loaded with data from the data store, which happens when it's state transitions from hollow to persistent-clean. It is called exactly after the object has consistent data in its default fetch group.

By implementing the `jdoPostLoad` method, the code to check for initial assignment of transient values can be saved, because JDO defines that this method is called only once, when an object is resolved. A class might implement the `jdoPostLoad` method to assign a value to a transient field, which depends on the value of a persistent field. This is especially useful for extensive calculations that should be done only once.

Another use case can be to register the persistent object with other objects of the application after it has been loaded from the data store. As an example, if deferring the initialization of a collection, time and memory can be saved until the object is actually used:

```
public class Book
    implements InstanceCallbacks
{
    transient Map myMap; // not initialized here!

    public Book(String title) { // used by application
        ...
    }

    private Book() { // used by JDO runtime ...
    }

    public void jdoPostLoad() {
        myMap = new HashMap(); // initialized when loaded
    }

    // other callbacks:
    public void jdoPreStore() {
    }
    public void jdoPreDelete() {
    }
    public void jdoPreClear() {
    }
}
```

4.4.2. Uses of jdoPreStore

The pre-store callback method is called before fields are written to the data store. One application of this callback is to update persistent fields with values of non-persistent fields before the transaction completes. Another idea can be to check values of persistent fields for consistent values or constraints and throw an exception if the rules don't allow that current values are to be stored.

Here is an example how to update a byte array field with a serialized stream of some non-persistent object:

```
public class Book
    implements InstanceCallbacks
{
    transient Color myColor; // not persistence-capable
    private byte[] array; // supported by most JDO
                           implementations

    public void jdoPostLoad() {
        ObjectInputStream in = new ObjectInputStream(
            new ByteArrayInputStream(array));
        myColor = (Color)in.readObject();
        in.close();
    }

    public void jdoPreStore() {
        ByteArrayOutputStream bo =
            new ByteArrayOutputStream();
        ObjectOutputStream out = new ObjectOutputStream(bo);
        out.writeObject(myColor);
        out.close();
        array = bo.getBytes();
    }
    // other callbacks:
    public void jdoPreDelete() {
    }
    public void jdoPreClear() {
    }
}
```

4.4.3. Uses of jdoPreDelete

This callback is invoked before an object is deleted from the data store, for instance when the application calls `PersistenceManager.deletePersistent`. It is neither called for objects that are deleted by other clients of the same database, nor for instances in other, parallel transactions. Instead, the `jdoPreDelete` callback should be used for in-memory instances only. A typical application for the callback is to delete dependent sub-objects:

```
public class BookShelf
    implements InstanceCallbacks
{
    // a BookShelf can contain either
    Collection shelves;
    Collection books;
    public void jdoPreDelete() {
        // get our PM:
        PersistenceManager pm =
            JDOHelper.getPersistenceManager(this);
        // delete shelves but not the books!
        pm.deleteAll(shelves);
    }
    // other callbacks:
    public void jdoPostLoad() {
    }
    public void jdoPreStore() {
    }
    public void jdoPreClear() {
    }
}
```

Long discussions came up in the JDO community, if the handling of dependent objects should be coded in the Java source or declared somewhere outside in the JDO XML file. In another use case the `jdoPreDelete` callback is responsible to deregister the object in the referencing object. This was already mentioned before. The coding might become quite complex, if the application's object model gets larger. Here is a little example:

```
public class Aisle
    implements InstanceCallbacks
{
    // an Aisle contains BookShelves:
    Collection shelves;
}

public class BookShelf
    implements InstanceCallbacks
```

```

{
    final Aisle aisle; // location of this shelf

    // can be created together:
    public BookShelf(Aisle ai) {
        this.aisle = ai;
        aisle.shelves.add(this);
    }

    private BookShelf() { /* JDO required */ }

    // on deletion, remove ourselves from
    // the enclosing object:
    public void jdoPreDelete() {
        aisle.shelves.remove(this);
    }

    // other callbacks:
    public void jdoPostLoad() {
    }
    public void jdoPreStore() {
    }
    public void jdoPreClear() {
    }
}

```

In JDO, it is always an option to model the relations between objects either as a forward reference (the Aisle has BookShelves) or as a backward reference (the BookShelf belongs to an Aisle). It is application specific and depends on the use cases, which option is taken, but when callbacks are used to de-register itself from the referencing object, a back pointer (Bookshelf.aisle) is necessary.

4.4.4. Uses of jdoPreClear

The jdoPreClear callback is called when an object becomes hollow or transient from persistent-deleted, persistent-new-deleted, persistent-clean or persistent-dirty during transaction termination. It should be used to clear non-persistent references to other objects to help the garbage collector, or to de-register it from other objects. It can also be used to clear transient fields, or to set them to illegal values, to prevent accidental access of cleared objects.

4.5 OPTIONAL STATES

JDO defines optional states and behaviour, which may be implemented by a JDO vendor. These states help the developer to use objects in an inconsistent manner, for instance outside of active transactions. The states can also be used for transient objects that should act like persistent objects in an active transaction. Before any such optional features are used by an application, it should check the JDO implementation's `PersistenceManagerFactory.supportedOptions()` method.

4.5.1. Transient Transactional

Transient instances of persistence-capable classes can become transactional, but not persistent in the data store by a call to `PersistenceManager.makeTransactional`. Those objects are called transient-transactional instances. Their data is somehow saved when the application calls `makeTransactional` and restored when the transaction rolls back, but no data is stored in the backend.

Transient-transactional is an optional JDO feature; nevertheless, most of the vendors implemented it at the time of this writing. To support lazy copy-on-write, a fresh transient-transactional instance enters the state `transient-clean` by a call to `makeTransactional`. Though its data is new or changed, this is only a marker for the persistence manager that any modification must be tracked. When the transient instance is modified, it becomes `transient-dirty` and a copy of its persistent fields is saved for later restore. There are two cases, when `makeTransactional` can be invoked: Before an active transaction begun or afterwards. In the first case the data of the object it had at the time the transaction started, will be saved. In the other case, the field data at the time `makeTransactional` was called will be saved.

Transient-dirty instances become `transient-clean` at either commit or rollback. During rollback, the previously saved values of managed fields are restored. Transient-clean instances can also be reset to non-transactional by invoking `makeNontransactional`. Both transient-clean and transient-dirty instances can be made persistent-new by a call to `makePersistent`.

The diagram shows the state transitions for transient-transactional objects:

4.5.2. Usages for transient-transactional instances

The basic idea for transient-transactional instances is transparency for the application. It is irrelevant whether an object is persistent or not, the state or data can be restored at rollback. An example is a GUI application that uses dialog boxes to display and edit configuration data. A dialog box may contain both persistent and local, transient configuration information. By using the transient-transactional feature, the implementation of the dialog box can simply rollback all data when "cancel" is clicked by the end-user.

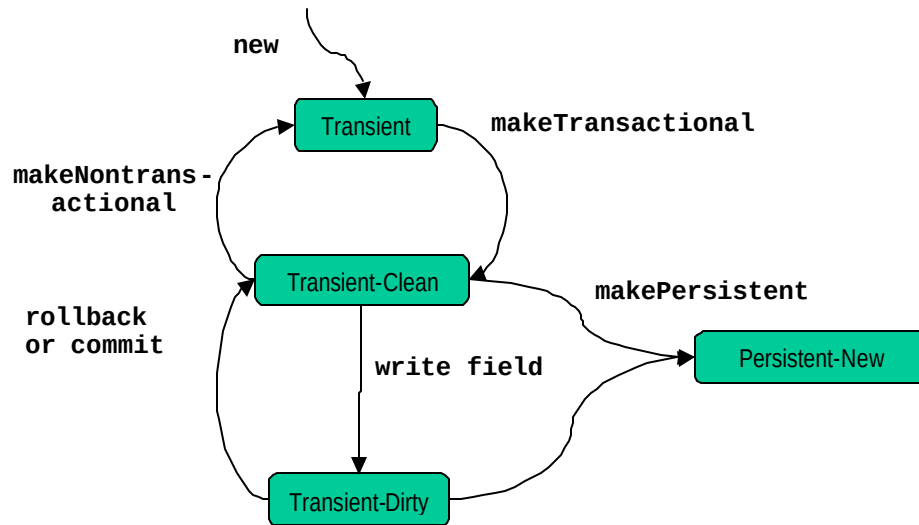


Figure 4-4 Life Cycle: Transient Transactional Instances

4.5.3. Nontransactional instances

The idea behind nontransactional instances is to support slow changing, inconsistent data and optimistic transactions. In some scenarios, an application wishes to read data explicitly into objects, but doesn't care for modifications by other clients of the same data store. This is often called isolation level 1, or dirty read semantics. Nontransactional instances can also be the result of terminated transactions.

JDO defines four different properties that change the behavior of the JDO object life cycle:

- NontransactionalRead
- NontransactionalWrite
- RetainValues
- Optimistic

The application has to set one or some of these properties to enable the specific options before the first use of a `PersistenceManager`. The reason for the options is safety: An application might modify an object by accident in a nontransactional context, but the modification gets lost or wrong data was read.

4.5.3.1. Nontransactional Read

This option allows an application to read fields of hollow or nontransactional instances outside of transactions. It also enables navigation and queries outside of active transactions. The state to support nontransactional, inconsistent field data outside of active transactions is called persistent-nontransactional. An application can force a persistent-clean object to become persistent-nontransactional by invoking `PersistenceManager.makeNontransactional`.

4.5.3.2. Nontransactional Write

With this option set to true, modification of nontransactional instances is allowed outside a transaction. Any changes to such objects are never written back to a data store though. JDO does not define how or when instances transition to persistent-nontransactional if they are modified outside a transaction, with `nontransactionalWrite` set to true.

4.5.3.3. Retain Values

JDO defines that all persistent instances are cleared when they become hollow to remove unwanted references and cleanup memory. With the `RetainValues` option set to true, persistent-clean, persistent-new, persistent-dirty objects become persistent-nontransactional instead of hollow at commit. The field data is kept for later use and it is allowed to read fields of such objects after the transaction terminated. Compared to Nontransactional Read, it is impossible to navigate to hollow objects and retrieve their data after transaction completion.

4.5.3.4. Restore Values

Without the `RestoreValues` option set to true, the above case is valid only if the transaction commits. At rollback instances still become hollow and they are unusable for the application. By setting the `RestoreValues` option to true, the JDO runtime must re-read the old state of persistent-dirty objects at rollback, or by other means restore the values of modified fields. All persistent-clean and persistent-dirty objects become persistent-nontransactional at rollback, making them readable by the application.

4.5.4. Usages for nontransactional instances

From an architectural standpoint, it is not really recommended to use nontransactional instances, because of consistency problems. The design guidelines for transaction handling

are discussed in chapter #####. On the other hand, there are special scenarios where nontransactional data has advantages:

- An application needs to display statistical data from time to time, and it is irrelevant whether the data is accurate at a specific point in time. Another design goal can be that the displaying part must not disturb the rest of the data processing. Under some circumstances, it might even be necessary to cache the displayed data and re-read only changes.
- A GUI application uses JDO for configuration data, which is loaded at a defined point in time by another part of the system. While editing the configuration data, transactional behavior is not needed. Only the change from one configuration set to another (new) one must be atomic, consistent, isolated and durable.

4.5.5. Optimistic transactions

The optional JDO optimistic transaction management changes the state transitions of persistent instances. *Optimistic* means a persistent instance can have transactional inconsistent values while a transaction is running, but at the end, when the transaction is committed, all previous assumptions are checked back against the data store. For this reason the state management has the option to let the application refresh data of some hollow or nontransactional instance before it is used inside the transaction.

When the application selects to use optimistic transactions, hollow instances transition to persistent-nontransactional instead of persistent-clean if a field is read. Upon updates of persistent fields, the instance transitions back to persistent-dirty like in pessimistic transactions. But the application has to explicitly invoke refresh before any update is made to retrieve fresh values from the data store. How optimistic transactions can be used will be explained in chapter ###.

4.5.5.1. When to use optimistic transactions

Actually any modification of some fields can lead to illegal data in the persistence manager's in-memory cache. Let's assume a data store has a table column defined to hold only unique values. A persistence manager that uses strict pessimistic transaction management would never allow to easily swapping two values, like in the following piece of code:

```
String temp = author1.name;  
author1.name = author2.name;  
author2.name = temp;
```

In the second line, the uniqueness constraint is hurt and a JDO implementation should detect this case. Fortunately, most JDO implementations are less strict and allow swapping two values in memory. In an optimistic transaction, the code runs fine and inconsistent in-memory values are expected. Later at commit, the persistence manager has to check either only modified or all values for consistent values and must throw an exception if expected data does not match the data in the data store.

Many people think that optimistic transactions also prevent locking problems, which isn't true. The problems are rather shifted in time until transaction commit.

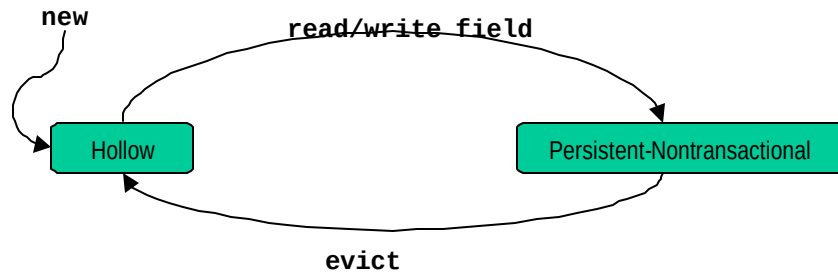


Figure 4-5 Life Cycle: Access outside transactions

4.6 PUTTING IT ALL TOGETHER

The following code excerpts show how to use the state interrogation methods and instance callbacks to debug common problems. A class is developed, which serves as a helper to find out about the actual state of objects, number of objects in memory, objects per state and so on. Since most JDO vendors do not provide methods to get information about a persistence manager's in-memory cache, it is quite handy to have a vendor-independent debug facility.

All the following methods can be found in the `DebugStates.java` source file.

The first method is a helper to get the memory id of an object. A very simple way to get a unique id of an object is to call `identityHashCode`. Though this is not a proof unique id, it works on common virtual machines and can help to debug objects in memory:

```
private static String memId(Object obj)
{
    long id = System.identityHashCode(obj);
    return Long.toString(id,16);
}
```

The next method returns a string that contains both the persistent object id and the in-memory hash code of an object. If the object is not yet persistent, just the memory id is returned:

```
private static String id(Object obj)
```

```

{
    if (obj instanceof PersistenceCapable) {
        PersistenceManager pm =
            JDOHelper.getPersistenceManager(obj);
        if (pm != null) {
            return "@"+memId(obj)+" OID:"+pm.getObjectId(obj);
        }
    }
    return "@"+memId(obj)+" no OID";
}

```

To get even more information about an object's life cycle, a table of counters is used to keep track of persistent instances. These counters are incremented in the InstanceCallback interface methods and the no-args constructor, which is used by the JDO implementation to create new hollow objects. A table entry object keeps a weak reference to each persistent instance, so that it is possible to get the number of objects that were garbage collected.

Here is a sample class, that uses the DebugStates class to count it's callback invocations:

```

public class Author
    implements InstanceCallbacks
{
    protected String name;

    public Author( String name ) {
        this.name = name;
    }

    public String toString()
    {
        return "Author: " + name;
    }

    /**
     * This is the constructor called for hollow objects.
     * It registers with the debug class.
     */
    private Author()
    {
        DebugStates.constructor(this);
    }

    // JDO callbacks, delegate to DebugStates:
    public void jdoPostLoad() {

```

```

        DebugStates.postLoad(this);
    }

    public void jdoPreStore() {
        DebugStates.preStore(this);
    }

    public void jdoPreClear() {
        DebugStates.preClear(this);
    }

    public void jdoPreDelete() {
        DebugStates.preDelete(this);
    }
}

```

The above DebugStates methods delegate to a function that gets a table entry object from a static table, based on the memory identity. If the entry does not exist, it is added to the table.

```

private static ObjectTable objectTable = new ObjectTable();

private static ObjectTableEntry tableEntry(Object obj)
{
    return objectTable.register(obj);
}

/**
 * Should be called in the no-args constructor of a
 * persistence-capable
 * class.
 */
public static void constructor(Object obj)
{
    tableEntry(obj).constructor++;
}

... equivalent methods for InstanceCallbacks ...

public ObjectTableEntry register(Object obj)
{
    Long id = new Long(System.identityHashCode(obj));
    ObjectTableEntry entry = (ObjectTableEntry)table.get(id);

```

```

    if (entry == null) {
        entry = new ObjectTableEntry(id,obj);
        table.put(id,entry);
    }
    return entry;
}

```

Now that every object is put into the table at some point in time, it is very easy to count used objects or find objects of a specific kind. The following method can be help to get some figures of the memory usage:

```

public String statistics()
{
    int constructors = 0;
    int postLoad = 0;
    int preStore = 0;
    int preClear = 0;
    int preDelete = 0;
    int objects = 0;
    int gced = 0;
    for (Iter i = new Iter();i.hasNext();) {
        ObjectTableEntry e = i.next();
        objects++;
        if (e.get() == null) gced++;
        constructors += e.constructor;
        postLoad += e.postLoad;
        preStore += e.preStore;
        preClear += e.preClear;
        preDelete += e.preDelete;
    }
    String NL = System.getProperty("line.separator");
    String result = "    Number of Objects:  "+objects+NL+
        "    Garbage collected:    "+gced+NL+
        "    Constructors called: "+constructors+NL+
        "    postLoad called:      "+postLoad+NL+
        "    preStore called:       "+preStore+NL+
        "    preClear called:       "+preClear+NL+
        "    preDelete called:      "+preDelete+NL;
    return result;
}

```

The result is somehow interesting, because even the first simple allocation of two objects like

```
pm.currentTransaction().begin();
```

```
Author a = new Author("Heiko Bobzin");
Book b = new Book("Core JDO", a);
```

prints the following result:

```
Number of Objects: 2
Garbage collected: 0
Constructors called: 2
postLoad called: 0
preStore called: 0
preClear called: 0
preDelete called: 0
```

Though the no-args constructor wasn't called in the sample code, it was traced that two other objects have been created. These two objects are held as factory objects by the JDO implementation to create new Author and Book instances. The code to call the no-args constructor has been added by the enhancer for security reasons.

When these two objects are made persistent (Author by reachability)

```
pm.makePersistent(b);
pm.currentTransaction().commit();
```

the preStore and preClear instance callbacks are called for each object:

```
Number of Objects: 2
Garbage collected: 0
Constructors called: 2
postLoad called: 0
preStore called: 2
preClear called: 2
preDelete called: 0
```

In a another test, all objects of the Book Extent were printed:

```
pm.currentTransaction().begin();
Extent e = pm.getExtent(Book.class, true);
Iterator iter = e.iterator();
while (iter.hasNext()) {
    Book a = (Book)iter.next();
    String title = a.title; // creates a hollow Author !
}
```

The database contained 20 Book instances and 20 Author instances, so that 42 objects were created in total (two for the factory objects). postLoad and preClear has been called for the Book instances only, whereas the other objects were hollow instances:


```
Number of Objects: 42
Garbage collected: 10
Constructors called: 42
postLoad called: 20
preStore called: 0
preClear called: 20
preDelete called: 0
```

4.7 SUMMARY

The JDO instance life cycle has been explained in this chapter to give an in-depth understanding of the dynamic aspects of persistent and transactional instances. The chapter has provided information about how and when to use state interrogation functions as well as callback methods. JDO optional states like *persistent nontransactional* and *transient transactional* were introduced and a commonly useable class to explore and trace JDO states has been developed.

With the understanding of the JDO object life cycle, it should be possible to drill into problems that are more complex and design JDO applications.

4.8 STATE TRANSITION TABLES AND DIAGRAMS

The following tables and diagrams show the complete state transitions and the operations that cause them.

Table 4-1 State Transitions

Current State	Transient	Persistent-new	Persistent-clean	Persistent-dirty	Hollow
Method					
makePersistent	persistent-new		unchanged		
deletePersistent	error!	persistent-new-deleted		persistent-deleted	
makeTransactional	transient-clean		unchanged		
makeNonTransactional	error!		persistent-nontrans.	error!	--
makeTransient	unchanged	error!	transient	error!	transient
commit	--		hollow		--
commit (retain values)	unchanged		persistent-nontrans.		unchanged
rollback	unchanged	transient	hollow		unchanged
rollback (retain values)	unchanged	transient	persistent-nontrans.		unchanged
refresh (active)	-----			persistent-clean	unchanged
refresh (optimistic)	unchanged	unchanged	unchanged	persistent-nontrans.	unchanged
evict	n/a	unchanged	hollow	unchanged	unchanged
read field (outside)	unchanged		impossible		persistent-nontrans.
read field (optimistic)		unchanged			persistent-nontrans.
read field (active)		unchanged			persistent-clean
write field (outside)	unchanged		impossible		persistent-nontrans.
write field (active)	unchanged		persistent-dirty	unchanged	persistent-dirty
retrieve (outside or optimistic)		unchanged			persistent-nontrans.
retrieve (active)		unchanged			persistent-clean

Table 4-2 State Transitions (cont.)

Current State	Transient-clean	Transient-dirty	Persistent-new-deleted	Persistent-deleted	Persistent-nontrans.
Method					
makePersistent	persistent-new			unchanged	
deletePersistent		error		unchanged	persistent-deleted
makeTransactional			unchanged		persistent-clean
makeNonTransactional	transient		error		unchanged
makeTransient	unchanged			error	transient
commit	unchanged	transient-clean		transient	
commit (retain values)	unchanged	transient-clean		transient	
rollback	unchanged	transient-clean	transient	hollow	unchanged
rollback (retain values)		transient-clean	transient	persistent-nontrans.	unchanged
refresh (active)			unchanged		
refresh (optimistic)			unchanged		
evict			unchanged		hollow
read field (outside)	unchanged		impossible		unchanged
read field (optimistic)	unchanged			error	unchanged
read field (active)	unchanged			error	persistent-clean
write field (outside)	unchanged		impossible		unchanged
write field (active)	transient-dirty	unchanged		error	persistent-dirty
retrieve (outside or optimistic)			unchanged		
retrieve (active)		unchanged			persistent-clean